

СОДЕРЖАНИЕ

1 Общая часть работы	4
1.1 Установка и подготовка среды	4
1.2 Исходный код	4
1.3 Получение AST	5
1.4 Генерация LLVM IR	6
1.5 Оптимизация IR	6
1.6 Граф потока управления программы	8
1.7 Выводы	9
2 Индивидуальная часть работы (по вариантам)	10
2.1 Комплексные числа	10
2.2 Структуры / записи	11
2.3 Списки / массивы / словари	13
2.4 Прототипы функций	14
2.5 Целочисленные константы	15
2.6 Форматирование / парсинг научной нотации	16
2.7 Вещественные константы	17
2.8 Строковые константы	18
2.9 Перечисления	19
2.10 Функции (объявление / создание)	20
2.11 Макросы с параметрами	21
2.12 Лямбда-выражения	22
2.13 Условные операторы	23
2.14 Тернарный оператор	24
2.15 Цикл do-while	25
2.16 Цикл while	26
2.17 Цикл for	27
Дополнительное задание. Локальные оптимизации синтаксических конструкций на промежуточном представлении	28

Лабораторная работа 7. Преобразование и анализ кода с использованием Clang и LLVM

Цель работы: Познакомиться с инструментами Clang и LLVM, научиться собирать AST и IR-промежуточное представление кода на C/C++, а также извлекать базовую информацию о программе (например, список функций).

Задачи:

1. Установить Clang и LLVM;
2. Скомпилировать простой C-файл с использованием clang и получить его: абстрактное синтаксическое дерево (AST), промежуточное представление LLVM IR;
3. Использовать opt для применения базовой комплексной оптимизации (например, O2);
4. Построить граф потока управления (CFG) для оптимизированной программы;
5. Проанализировать результат, сделать выводы и ответить на контрольные вопросы.
6. Выполнить индивидуальное задание в соответствии со своим оператором из КР / РГР.

Список контрольных вопросов:

1. Что такое Clang, и какова его роль в процессе компиляции программ?
2. Что представляет собой LLVM и как он используется в современных компиляторах?
3. Чем отличается абстрактное синтаксическое дерево (AST) от промежуточного представления LLVM IR?
4. Для чего необходимо промежуточное представление (IR) в процессе компиляции?

5. Что делает инструкция alloc в LLVM IR, и зачем она используется в функциях?
6. Зачем нужна оптимизация кода в компиляторе, и какие основные цели она преследует?
7. Что такое SSA-форма и почему она важна при оптимизации программ?
8. Что такое граф потока управления (CFG) и как он помогает анализировать поведение программы?
9. Как устроено представление арифметических операций в LLVM IR (например, умножение, сложение)?
10. Почему функции в LLVM IR обычно представляют собой отдельные единицы анализа и оптимизации?
11. Что происходит с функцией в LLVM IR, если она вызывается один раз и очень короткая?
12. Какие преимущества даёт использование IR и CFG для автоматических оптимизаций по сравнению с анализом исходного текста на C?

1 Общая часть работы

1.1 Установка и подготовка среды

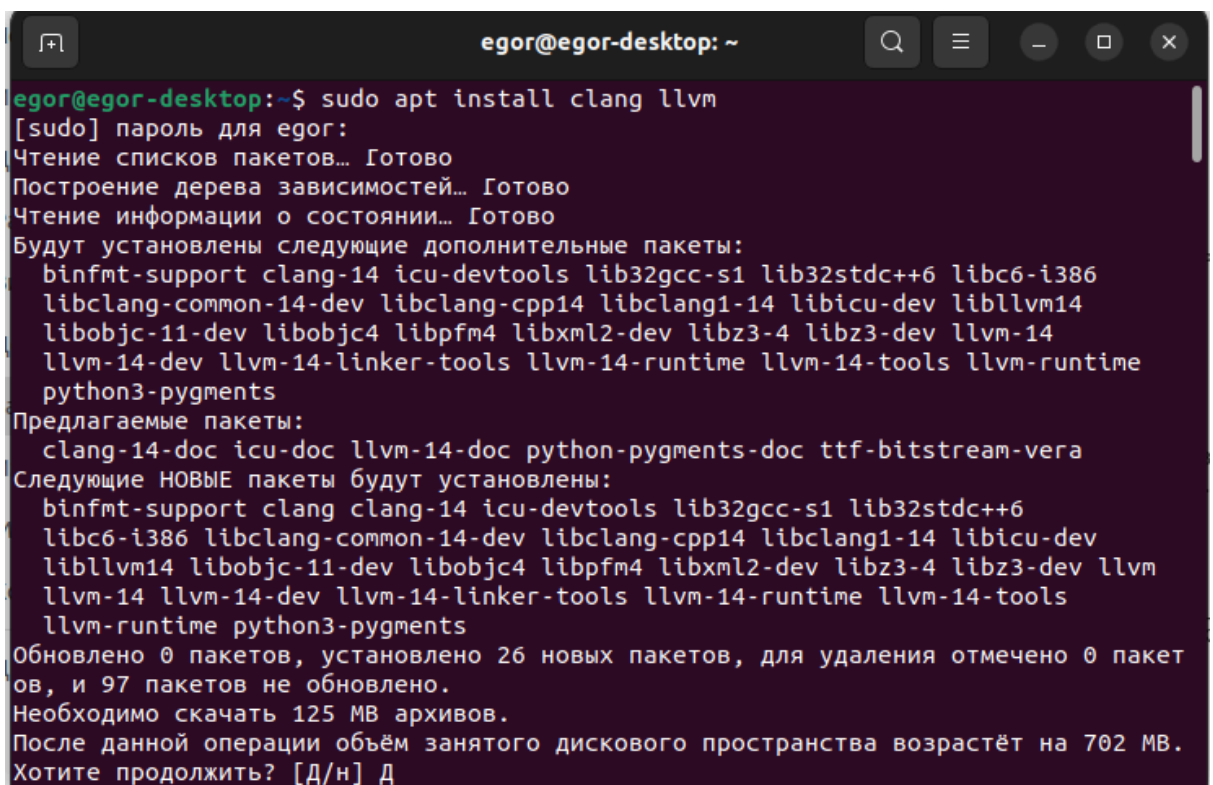
Работа выполнялась в среде Ubuntu 22.04. Установлены следующие инструменты:

- clang — компилятор языка C/C++;
- llvm — инструменты анализа и оптимизации кода;
- opt — инструмент для работы с LLVM IR и применения

оптимизаций;

- Graphviz — инструмент для визуализации кода.

Команда установки: `sudo apt install clang llvm`



```
egor@egor-desktop: ~  
egor@egor-desktop:~$ sudo apt install clang llvm  
[sudo] пароль для egor:  
Чтение списков пакетов... Готово  
Построение дерева зависимостей... Готово  
Чтение информации о состоянии... Готово  
Будут установлены следующие дополнительные пакеты:  
  binfmt-support clang-14 icu-devtools lib32gcc-s1 lib32stdc++6 libc6-i386  
  libclang-common-14-dev libclang-cpp14 libclang1-14 libicu-dev libllvm14  
  libobjc-11-dev libobjc4 libpfm4 libxml2-dev libz3-4 libz3-dev llvm-14  
  llvm-14-dev llvm-14-linker-tools llvm-14-runtime llvm-14-tools llvm-runtime  
  python3-pygments  
Предлагаемые пакеты:  
  clang-14-doc icu-doc llvm-14-doc python-pygments-doc ttf-bitstream-vera  
Следующие НОВЫЕ пакеты будут установлены:  
  binfmt-support clang clang-14 icu-devtools lib32gcc-s1 lib32stdc++6  
  libc6-i386 libclang-common-14-dev libclang-cpp14 libclang1-14 libicu-dev  
  libllvm14 libobjc-11-dev libobjc4 libpfm4 libxml2-dev libz3-4 libz3-dev llvm  
  llvm-14 llvm-14-dev llvm-14-linker-tools llvm-14-runtime llvm-14-tools  
  llvm-runtime python3-pygments  
Обновлено 0 пакетов, установлено 26 новых пакетов, для удаления отмечено 0 пакет  
ов, и 97 пакетов не обновлено.  
Необходимо скачать 125 МВ архивов.  
После данной операции объём занятого дискового пространства возрастёт на 702 МВ.  
Хотите продолжить? [д/н] д
```

1.2 Исходный код

Программа на языке C:

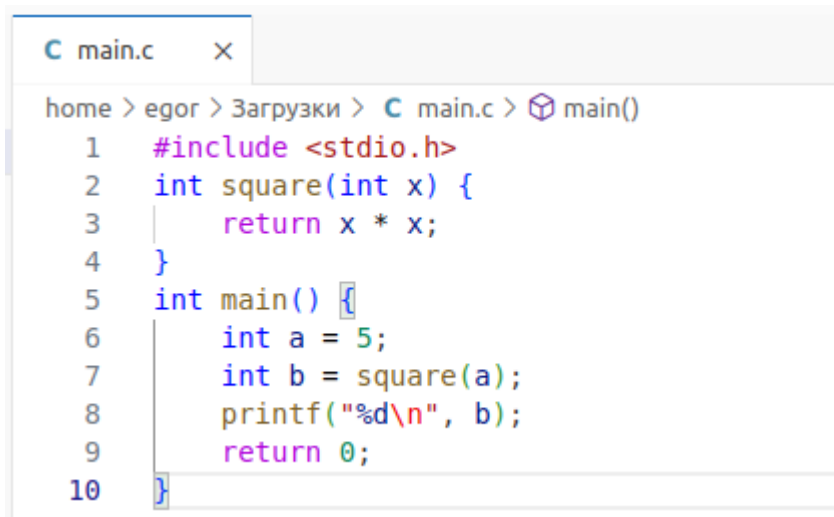
```
#include <stdio.h>  
  
int square(int x) {  
    return x * x;  
}  
  
int main() {
```

```

    int a = 5;
    int b = square(a);
    printf("%d\n", b);
    return 0;
}

```

Сохранена в файл main.c.



```

C main.c x
home > egor > Загрузки > C main.c > main()
1  #include <stdio.h>
2  int square(int x) {
3      return x * x;
4  }
5  int main() {
6      int a = 5;
7      int b = square(a);
8      printf("%d\n", b);
9      return 0;
10 }

```

1.3 Получение AST

Команда: clang -Xclang -ast-dump -fsyntax-only main.c



```

-FunctionDecl 0x2c257d50 <main.c:2:1, line:4:1> line:2:5 used square 'int (int)'
-ParamVarDecl 0x2c257cb8 <col:12, col:16> col:16 used x 'int'
-CompoundStmt 0x2c257e98 <col:19, line:4:1>
  -ReturnStmt 0x2c257e88 <line:3:5, col:16>
    -BinaryOperator 0x2c257e68 <col:12, col:16> 'int' '*'
      -ImplicitCastExpr 0x2c257e38 <col:12> 'int' <LValueToRValue>
        -DeclRefExpr 0x2c257df8 <col:12> 'int' lvalue ParmVar 0x2c257cb8 'x' 'int'
      -ImplicitCastExpr 0x2c257e50 <col:16> 'int' <LValueToRValue>
        -DeclRefExpr 0x2c257e18 <col:16> 'int' lvalue ParmVar 0x2c257cb8 'x' 'int'
-FunctionDecl 0x2c257f00 <line:5:1, line:10:1> line:5:5 main 'int ()'
-CompoundStmt 0x2c258868 <col:12, line:10:1>
  -DeclStmt 0x2c258040 <line:6:5, col:14>
    -VarDecl 0x2c257fb8 <col:5, col:13> col:9 used a 'int' cinit
      -IntegerLiteral 0x2c258020 <col:13> 'int' 5
  -DeclStmt 0x2c2581a0 <line:7:5, col:22>
    -VarDecl 0x2c258070 <col:5, col:21> col:9 used b 'int' cinit
      -CallExpr 0x2c258160 <col:13, col:21> 'int'
        -ImplicitCastExpr 0x2c258148 <col:13> 'int (*) (int)' <FunctionToPointerDecay>
          -DeclRefExpr 0x2c2580d8 <col:13> 'int (int)' Function 0x2c257d50 'square' 'int (int)'
        -ImplicitCastExpr 0x2c258188 <col:20> 'int' <LValueToRValue>
          -DeclRefExpr 0x2c2580f8 <col:20> 'int' lvalue Var 0x2c257fb8 'a' 'int'
      -CallExpr 0x2c2587c0 <line:8:5, col:21> 'int'
        -ImplicitCastExpr 0x2c2587a8 <col:5> 'int (*) (const char *, ...)' <FunctionToPointerDecay>
          -DeclRefExpr 0x2c2581b8 <col:5> 'int (const char *, ...)' Function 0x2c23cd28 'printf' 'int (const char *, ...)'
        -ImplicitCastExpr 0x2c258808 <col:12> 'const char *' <NoOp>
          -ImplicitCastExpr 0x2c2587f0 <col:12> 'char *' <ArrayToPointerDecay>
            -StringLiteral 0x2c2581d8 <col:12> 'char[4]' lvalue "%d\n"
          -ImplicitCastExpr 0x2c258820 <col:20> 'int' <LValueToRValue>
            -DeclRefExpr 0x2c2581f8 <col:20> 'int' lvalue Var 0x2c258070 'b' 'int'
      -ReturnStmt 0x2c258858 <line:9:5, col:12>
        -IntegerLiteral 0x2c258838 <col:12> 'int' 0

```

Функция square принята, содержит параметр x и возвращает x * x.

1.4 Генерация LLVM IR

Команда: `clang -S -emit-llvm main.c -o main.ll`



```
1; ModuleID = 'main.c'
2 source_filename = "main.c"
3 target_datalayout = "e-nie-p270:32:32-p271:32:32-p272:64:64-164:64-f80:128-n8:16:32:64-S128"
4 target_triple = "x86_64-pc-linux-gnu"
5
6 @.str = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
7
8; Function Attrs: noinline nounwind optnone uwtable
9 define dso_local i32 @square(i32 noundef %0) #0 {
10     %2 = alloca i32, align 4
11     store i32 %0, i32* %2, align 4
12     %3 = load i32, i32* %2, align 4
13     %4 = load i32, i32* %2, align 4
14     %5 = mul nsw i32 %3, %4
15     ret i32 %5
16 }
17
18; Function Attrs: noinline nounwind optnone uwtable
19 define dso_local i32 @main() #0 {
20     %1 = alloca i32, align 4
21     %2 = alloca i32, align 4
22     %3 = alloca i32, align 4
23     store i32 0, i32* %1, align 4
24     store i32 5, i32* %2, align 4
25     %4 = load i32, i32* %2, align 4
26     %5 = call i32 @square(i32 noundef %4)
27     store i32 %5, i32* %3, align 4
28     %6 = load i32, i32* %3, align 4
29     %7 = call i32 (i8*, ...) @printf(i8* noundef getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0, i64 0), i32 noundef %6)
30     ret i32 0
31 }
32
33 declare i32 @printf(i8* noundef, ...) #1
34
35 attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
36 attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
37
38 !llvm.module.flags = !{!0, !1, !2, !3, !4}
39 !llvm.ident = !{!5}
40
41 !0 = !{i32 1, !"wchar_size", i32 4}
42 !1 = !{i32 7, !"PIC Level", i32 2}
43 !2 = !{i32 7, !"PIE Level", i32 2}
44 !3 = !{i32 7, !"uwtable", i32 1}
45 !4 = !{i32 7, !"frame-pointer", i32 2}
46 !5 = !{!"Ubuntu clang version 14.0.0-1ubuntu1.1"}
```

1.5 Оптимизация IR

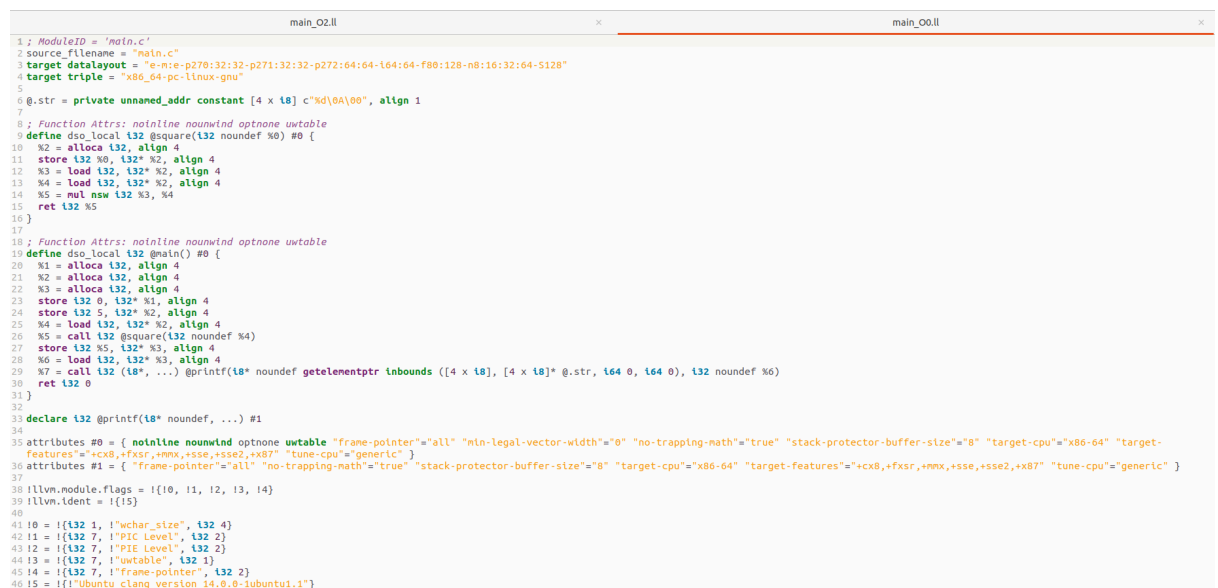
Команда: `clang -O0 -S -emit-llvm main.c -o main_O0.ll`

Стоит отметить, что в файле с IR до оптимизации:

Все переменные (a, b, x.addr) размещены в памяти через `alloca`;

Множество операций `load` и `store`;

`square` вызывается как отдельная функция.



```
1; ModuleID = 'main.c'
2 source_filename = "main.c"
3 target_datalayout = "e-nie-p270:32:32-p271:32:32-p272:64:64-164:64-f80:128-n8:16:32:64-S128"
4 target_triple = "x86_64-pc-linux-gnu"
5
6 @.str = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
7
8; Function Attrs: noinline nounwind optnone uwtable
9 define dso_local i32 @square(i32 noundef %0) #0 {
10     %2 = alloca i32, align 4
11     store i32 %0, i32* %2, align 4
12     %3 = load i32, i32* %2, align 4
13     %4 = load i32, i32* %2, align 4
14     %5 = mul nsw i32 %3, %4
15     ret i32 %5
16 }
17
18; Function Attrs: noinline nounwind optnone uwtable
19 define dso_local i32 @main() #0 {
20     %1 = alloca i32, align 4
21     %2 = alloca i32, align 4
22     %3 = alloca i32, align 4
23     store i32 0, i32* %1, align 4
24     store i32 5, i32* %2, align 4
25     %4 = load i32, i32* %2, align 4
26     %5 = call i32 @square(i32 noundef %4)
27     store i32 %5, i32* %3, align 4
28     %6 = load i32, i32* %3, align 4
29     %7 = call i32 (i8*, ...) @printf(i8* noundef getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0, i64 0), i32 noundef %6)
30     ret i32 0
31 }
32
33 declare i32 @printf(i8* noundef, ...) #1
34
35 attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
36 attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
37
38 !llvm.module.flags = !{!0, !1, !2, !3, !4}
39 !llvm.ident = !{!5}
40
41 !0 = !{i32 1, !"wchar_size", i32 4}
42 !1 = !{i32 7, !"PIC Level", i32 2}
43 !2 = !{i32 7, !"PIE Level", i32 2}
44 !3 = !{i32 7, !"uwtable", i32 1}
45 !4 = !{i32 7, !"frame-pointer", i32 2}
46 !5 = !{!"Ubuntu clang version 14.0.0-1ubuntu1.1"}
```

Команда: `clang -O2 -S -emit-llvm main.c -o main_O2.ll`

Команда -O2 – комплексная оптимизация среднего уровня. Она применяет более 30 различных оптимизаций:

- -inline – встраивание небольших функций (встраивает square в main, если она вызывается один раз);
- -constprop – подставит значение square(5) → 25, если функция встроена и всё известно на этапе компиляции;
- -mem2reg – перевод переменных из памяти в регистры (SSA);
- -instcombine – объединение и упрощение инструкций (упростит арифметику, например $x * x$ может быть преобразовано в shl при $x = 2^n$);
- -simplifycfg – оптимизирует структуру блоков (Упростит граф управления, если после inlining останутся лишние блоки);
- -reassociate, -gvn, -sroa, -dce и другие.

В файле с IR после оптимизации:

Вся функция square исчезла – она была встроена (-inline) и затем вычислена (оптимизация -constprop);

Никаких переменных, alloca, store, load – всё удалено (оптимизации -mem2reg, -dce);

Остался только вызов printf(25).

```
main_O2.ll                                     ×                                     main_O0.ll                                     ×
1; ModuleID = 'main.c'
2 source_filename = "main.c"
3 target datalayout = "e-nie-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
4 target triple = "x86_64-pc-linux-gnu"
5
6 @.str = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1
7
8; Function Attrs: mustprogress norecurse nosync nounwind readnone uwtable willreturn
9 define dso_local @square(i32 @noundef %0) local_unnamed_addr #0 {
10     %2 = mul nsw i32 %0, %0
11     ret i32 %2
12 }
13
14; Function Attrs: norecurse nounwind uwtable
15 define dso_local @main() local_unnamed_addr #1 {
16     %1 = tail call @square(i32 @, ...) @printf(i8* noundef nonnull dereferenceable(1) @.str, i64 @, i64 @, i32 noundef 25)
17     ret i32 0
18 }
19
20; Function Attrs: norecurse nounwind
21 declare noundef @printf(i8* nocapture noundef readonly, ...) local_unnamed_addr #2
22
23 attributes #0 = { mustprogress norecurse nosync nounwind readnone uwtable willreturn "frame-pointer"="none" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
24 attributes #1 = { norecurse nounwind uwtable "frame-pointer"="none" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
25 attributes #2 = { norecurse nounwind "frame-pointer"="none" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
26
27 !llvm.module.flags = !{!0, !1, !2, !3}
28 !llvm.ident = !{!4}
29
30 !0 = !{i32 1, !"wchar_size", i32 4}
31 !1 = !{i32 7, !"PIC Level", i32 2}
32 !2 = !{i32 7, !"PIE Level", i32 2}
33 !3 = !{i32 7, !"uwtable", i32 1}
34 !4 = !{! "Ubuntu clang version 14.0.0-1ubuntu1.1"}
```

Команда: diff main_O0.ll main_O2.ll

Сравнение двух файлов:

```

egor@egor-desktop:~$ clang -O0 -S -emit-llvm main.c -o main_00.ll
egor@egor-desktop:~$ clang -O2 -S -emit-llvm main.c -o main_02.ll
egor@egor-desktop:~$ diff main_00.ll main_02.ll
8,15c8,11
< ; Function Attrs: noinline nounwind optnone uwtable
< define dso_local @square(i32 @noundef %0) #0 {
<   %2 = alloca i32, align 4
<   store i32 %0, i32* %2, align 4
<   %3 = load i32, i32* %2, align 4
<   %4 = load i32, i32* %2, align 4
<   %5 = mul nsw i32 %3, %4
<   ret i32 %5
< ---
> ; Function Attrs: mustprogress norecurse nosync nounwind readnone uwtable willreturn
> define dso_local @square(i32 @noundef %0) local_unnamed_addr #0 {
>   %2 = mul nsw i32 %0, %0
>   ret i32 %2
18,29c14,16
< ; Function Attrs: noinline nounwind optnone uwtable
< define dso_local @square(i32 @noundef %0) #0 {
<   %1 = alloca i32, align 4
<   %2 = alloca i32, align 4
<   %3 = alloca i32, align 4
<   store i32 0, i32* %1, align 4
<   store i32 5, i32* %2, align 4
<   %4 = load i32, i32* %2, align 4
<   %5 = call @square(i32 @noundef %4)
<   store i32 %5, i32* %3, align 4
<   %6 = load i32, i32* %3, align 4
<   %7 = call @square(i32 @noundef %6)
<   @printf(i8* @noundef getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0, i64 0), i32 @noundef %6)
< ---
> ; Function Attrs: norecurse nounwind uwtable
> define dso_local @main() local_unnamed_addr #1 {
>   %1 = tail call @square(i32 @noundef %0) @printf(i8* @noundef nonnull dereferenceable(1) getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0, i64 0), i32 @noundef 25)
33c20,21
< declare i32 @printf(i8* @noundef, ...) #1
< ---
> ; Function Attrs: norecurse nounwind
> declare @noundef i32 @printf(i8* @noundef nocapture noundef readonly, ...) local_unnamed_addr #2
35,36c23,25
< attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
< attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
< ---
> attributes #0 = { mustprogress norecurse nosync nounwind readnone uwtable willreturn "frame-pointer"="none" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
> attributes #1 = { norecurse nounwind uwtable "frame-pointer"="none" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
> attributes #2 = { norecurse nounwind "frame-pointer"="none" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

```

Стоит отметить, что после оптимизации произошли следующие изменения:

- Переменные типа аллоса были удалены;
- Код переведён в SSA-форму;
- Оптимизация улучшила читаемость и упростила поток

управления.

1.6 Граф потока управления программы

Команда для генерации оптимизированного LLVM IR: `clang -O2 -S -emit-llvm main.c -o main.ll`

Команда для генерации .dot-файлов CFG для функций: `opt -dot-cfg -disable-output main.ll`

```

egor@egor-desktop:~$ opt -dot-cfg -disable-output main.ll
egor@egor-desktop:~$ find . -name "*.dot"
./main.dot
./square.dot

```

Эта команда создаст DOT-файлы: `main.dot` – для функции `main`; `square.dot` – для `square`, если она не была удалена оптимизацией.

Команда для установки библиотеки Graphviz: `sudo apt install graphviz`

Команды для преобразования файлов с расширением .dot в .png с помощью Graphviz:

```
dot -Tpng .main.dot -o cfg_main.png
```

```
dot -Tpng .square.dot -o cfg_square.png
```

Команды для просмотра файлов с CGF:

```
xdg-open cfg_main.png
```

```
%0:  
%1 = tail call i32 @printf(i8* noundef nonnull dereferenceable(1)  
... getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0, i64 0), i32 noundef  
... 25)  
ret i32 0
```

CFG for 'main' function

```
xdg-open cfg_square.png
```

```
%1:  
%2 = mul nsw i32 %0, %0  
ret i32 %2
```

CFG for 'square' function

Стоит отметить, что в LLVM каждый граф потока управления (CFG) строится на уровне функции, поскольку структура управления всегда локальна для тела функции. Для получения полного представления о программе, нужно построить CFG для всех функций и анализировать их совокупность. Автоматическое объединение всех CFG в один граф не предусмотрено в LLVM по умолчанию.

1.7 Выводы

- С помощью Clang можно получить полную структуру AST и IR, а также CGF;
- LLVM предоставляет гибкие инструменты анализа и оптимизации;
- Промежуточное представление кода удобно для написания компиляторных трансформаций.

2 Индивидуальная часть работы (по вариантам)

2.1 Комплексные числа

Симоняк Семён Вячеславович, АП-326

Чернышев Родион Вячеславович, АВТ-314

Тарбаев Даба-Цырен Батоевич, АП-327

Кокунов Андрей Алексеевич, АВТ-313

Лузянин Захар Сергеевич, АП-326

Васильев Антон Романович, АВТ-314

Пример кода:

```
#include <iostream>
#include <complex>

int main() {
    std::complex<double> z1(3.0, 4.0);
    std::complex<double> z2(1.0, 2.0);
    auto z3 = z1 * z2 + z1;
    std::cout << z3.real() << " " << z3.imag() << std::endl;
    return 0;
}
```

Задания:

1. Постройте AST и LLVM IR (без оптимизаций).
2. Примените оптимизации -O1, -O2, -O3 и сравните полученный IR.
3. Исследуйте, встраивается ли оператор * и + для комплексных чисел.
4. Постройте CFG для функции main до и после -O2.
5. Сделайте выводы о том, как компилятор обрабатывает пользовательские операторы.

2.2 Структуры / записи

Бабаева Дарья Евгеньевна, АП-326

Осинцева Дарья Александровна, АП-327

Заозернов Виталий Анатольевич, АВТ-314

Шевякова Анастасия Игоревна, АВТ-314

Зоркольников Илья Алексеевич, АВТ-313

Кузьмин Илья Андреевич, АП-326

Зохно Даниил Русланович, АП-326

Башинов Арья Игоревич, АП-326

Пример кода:

```
#include <stdio.h>

struct Point {
    int x;
    int y;
};

int sum(struct Point p) {
    return p.x + p.y;
}

int main() {
    struct Point p = {2, 3};
    int result = sum(p);
    printf("%d\n", result);
    return 0;
}
```

Задания:

1. Получите AST и LLVM IR.
2. Примените -O2 и опишите, изменилась ли передача структуры (по значению / по ссылке?).
3. Постройте CFG для sum и main.

4. Исследуйте, что происходит, если добавить `__attribute__((always_inline))` к `sum`.
5. Вывод: как LLVM оптимизирует передачу структур?

2.3 Списки / массивы / словари

Бикмухаметов Таир Олегович, АВТ-314

Костоломов Александр Евгеньевич, АВТ-314

Комиссарова Юлия Викторовна, АП-326

Головина Альбина Сергеевна, АП-327

Зайцев Александр Анатольевич, АВТ-313

Мосина Виктория Владимировна, АВТ-314

Новохатский Никита Сергеевич, АВТ-313

Гордеев Андрей Владимирович, АП-326

Лабузова Виктория Витальевна, АВТ-314

Старченко Дмитрий Евгеньевич, АВТ-313

Пример кода:

```
#include <iostream>
#include <array>

int main() {
    std::array<int, 5> data = {1, 2, 3, 4, 5};
    int sum = 0;
    for (int i = 0; i < data.size(); ++i) {
        sum += data[i];
    }
    std::cout << sum << std::endl;
    return 0;
}
```

Задания:

1. Получите IR для -O0 и -O2.
2. Исследуйте, разворачивается ли цикл (-unroll).
3. Постройте CFG для main с -O0 и с -O2.
4. Примените дополнительно -loop-rotate, -licm и опишите изменения.
5. Вывод: какие оптимизации применились к массиву и циклу?

2.4 Прототипы функций

Косяченко Даниил Алексеевич, АВТ-313

Ковалев Егор Евгеньевич, АП-327

Попов Виктор Андреевич, АВТ-314

Амельченко Семен Андреевич, АП-326

Пример кода:

```
int sum(int a, int b); // прототип
```

```
int main() {  
    return sum(5, 7);  
}
```

```
int sum(int a, int b) {  
    return a + b;  
}
```

Задания:

1. Постройте AST. Укажите, виден ли на нем прототип.
2. Получите IR без оптимизаций.
3. Примените -O2. Произошло ли встраивание sum?
4. Постройте CFG для main и sum.
5. Исследуйте, что изменится при добавлении static к sum.
6. Сделайте выводы о роли прототипа в оптимизациях.

2.5 Целочисленные константы

Арифуллин Алан Дмитриевич, АП-327

Коробейникова Дарья Романовна, АП-326

Сорочинский Михаил Владимирович, АВТ-314

Геращенко Антон Евгеньевич, АВТ-313

Зимин Бэлигто Очирович, АВТ-314

Марченко Александр Евгеньевич, АП-326

Пример кода:

```
const int LIMIT = 100;
```

```
int main() {  
    int sum = 0;  
    for (int i = 0; i < LIMIT; ++i) {  
        sum += i;  
    }  
    return sum;  
}
```

Задания:

1. Получите IR для -O0.
2. Примените -O2 и найдите, исчезла ли переменная LIMIT.
3. Примените отдельно -constprop и -ipsccp.
4. Постройте CFG до и после оптимизаций.
5. Сделайте вывод о том, как и когда константа подставляется?

2.6 Форматирование / парсинг научной нотации

Базыкина Диана Александровна, АП-326

Попова Дарья Алексеевна, АВТ-314

Петрухно Владимир Константинович, АП-327

Соболев Илья Олегович, АВТ-313

Пример кода:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    const char* str = "3.14e2";
    double value = strtod(str, NULL);
    printf("%f\n", value);
    return 0;
}
```

Задания:

1. Получите AST и IR.
2. Примените -O2 и проверьте, вызывается ли strtod или она встроена.
3. Исследуйте, можно ли заменить вызов на константу (если строка известна).
4. Постройте CFG.
5. Сделайте вывод о том, как LLVM обрабатывает функции парсинга?

2.7 Вещественные константы

Шипунов Дмитрий Николаевич, АП-326

Олейник Егор Викторович, АП-326

Коченков Андрей Артемович, АП-327

Гидульянов Кирилл Сергеевич, АВТ-313

Марков Данил Дмитриевич, АВТ-314

Пример кода:

```
const double PI = 3.141592653589793;
```

```
int main() {  
    double r = 2.0;  
    double area = PI * r * r;  
    return (int)area;  
}
```

Задания:

1. Получите IR для -O0.
2. Получите IR для -O2. Произошло ли свертывание константы?
3. Примените -constprop, -globalopt, -ipsccp.
4. Сравните CFG.
5. Сделайте вывод о том, когда вещественная константа

вычисляется на этапе компиляции?

2.8 Строковые константы

Пузырный Даниил Алексеевич, АП-327

Дарчук Софья Сергеевна, АП-326

Ткачук Фёдор Андреевич, АВТ-314

Обеленец Павел Викторович, АВТ-313

Дядов Владислав Леонидович, АВТ-314

Дементьев Артём Андреевич, АВТ-313

Пример кода:

```
#include <stdio.h>
```

```
int main() {  
    const char* msg = "Hello, World!";  
    printf("%s\n", msg);  
    return 0;  
}
```

Задания:

1. Получите AST и IR для -O0.
2. Примените -O2 и -mergeconst.
3. Исследуйте, объединяются ли одинаковые строки.
4. Постройте CFG.
5. Сделайте вывод о том, как LLVM управляет строковыми литералами.

2.9 Перечисления

Адодин Петр Юрьевич, АП-327

Данькин Дмитрий Витальевич, АВТ-314

Агаев Атамырат, АВТ-314

Еловикова Кристина Сергеевна, АВТ-313

Седельников Александр Алексеевич, АП-326

Семешко Анастасия Николаевна, АП-325

Пример кода:

```
enum Color { RED, GREEN, BLUE };
```

```
int main() {  
    enum Color c = GREEN;  
    int val = c + 10;  
    return val;  
}
```

Задания:

1. Получите AST. Как в нем представлено перечисление?
2. Получите IR для -O0 и -O2. Исследуйте, заменяется ли GREEN на 1 сразу?
3. Постройте CFG.
4. Сделайте вывод о том, как перечисления влияют на IR? Не являются ли они “синтаксическим сахаром”?

2.10 Функции (объявление / создание)

Ненашкин Владимир Денисович, АВТ-313

Савченко Владислав Эдуардович, АВТ-313

Костюк Кирилл Яковлевич, АВТ-314

Болтенков Евгений Олегович, АВТ-314

Коваленко Евгений Алексеевич, АП-327

Мавлонхонов Жобирхон Сайдуллохон угли, АП-326

Пример кода:

```
inline int square(int x) {  
    return x * x;  
}
```

```
int main() {  
    int a = 5;  
    int b = square(a);  
    return b;  
}
```

Задания:

1. Получите IR для -O0.
2. Получите IR для -O2. Встроилась ли функция?
3. Примените -always-inline и сравните с предыдущими оптимизациями.
4. Постройте CFG до и после.
5. Сделайте вывод об условиях встраивания функций в LLVM.

2.11 Макросы с параметрами

Парамонова Кристина Федоровна, АВТ-314

Пример кода:

```
#define SQUARE(x) ((x) * (x))
```

```
int main() {  
    int a = 5;  
    int b = SQUARE(a + 1);  
    return b;  
}
```

Задания:

1. Постройте AST. Укажите, виден ли макрос?
2. Получите IR для -O0.
3. Получите IR для -O2. Вычислено ли выражение на этапе компиляции?
4. Постройте CFG.
5. Сделайте вывод: отличие макроса от inline-функции (задание 2.10.1) в IR.

2.12 Лямбда-выражения

Ситников Владислав Игоревич, АП-326

Гусейнов Рза Анар Оглы, АВТ-314

Орехов Илья Сергеевич, АП-326

Гетман Денис Андреевич, АВТ-314

Плахин Даниил Александрович, АП-327

Ижболдина Виолетта Вячеславовна, АВТ-313

Герасимов Сергей Павлович, АВТ-313

Пример кода:

```
int main() {  
    auto square = [](int x) -> int { return x * x; };  
    int a = 5;  
    int b = square(a);  
    return b;  
}
```

Задания:

1. Получить AST, указав как представлена лямбда-функция.
 2. Получить IR для -O0 и найти замыкание.
 3. Получить IR для -O2 и указать, во что превратилась лямбда-функция?
 4. Постройте CFG.
 5. Вывод: лямбда — синтаксический сахар или отдельный тип.
- Сделайте вывод о том, являются ли лямбда-выражения отдельным типом или “синтаксическим сахаром”?

2.13 Условные операторы

Топоев Максим Александрович, АП-327

Горащенко Дарья Романовна, АВТ-313

Леонтьев Александр Вячеславович, АВТ-314

Пример кода:

```
int max(int a, int b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

Задания:

1. Построить AST и IR для -O0.
2. Примените -O2. Изменилось ли условие на `stov`?
3. Постройте CFG.
4. Исследуйте, меняется ли CFG при использовании `-branch-prob`.
5. Сделайте вывод о том, как LLVM оптимизирует условные переходы.

2.14 Тернарный оператор

Геронимус Матвей Анатольевич, АП-326

Сущих Анна Александровна, АП-326

Пример кода:

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

Задания:

1. Сравните AST и IR с if-else (задание 2.13.1).
2. Примените -O2 — получили ли `mov`?
3. Постройте CFG (сравните с if-else, задание 2.13.3).
4. Сделайте вывод о различиях тернарного оператора и условных переходов в IR.

2.15 Цикл do-while

Вейбер Александр Сергеевич, АП-327

Лантух Елизавета Евгеньевна, АВТ-313

Дудовцев Игорь Игоревич, АВТ-314

Пример кода:

```
int main() {  
    int i = 0, sum = 0;  
    do {  
        sum += i;  
        i++;  
    } while (i < 10);  
    return sum;  
}
```

Задания:

1. Получите IR для -O0.
2. Получите IR для -O2. Какие оптимизации применены?
3. Примените -loop-unroll и -loop-rotate.
4. Постройте CFG.
5. Укажите в чем отличие do-while от while в IR (задание 2.16.1).

2.16 Цикл while

Кашин Андрей Вячеславович, АВТ-314

Маст Денис Евгеньевич, АВТ-313

Федулова Виктория Валерьевна, АВТ-313

Гладышев Роман Миртагиевич, АВТ-314

Пример кода:

```
int main() {  
    int i = 0, sum = 0;  
    while (i < 10) {  
        sum += i;  
        i++;  
    }  
    return sum;  
}
```

Задания:

1. Постройте IR для -O0.
2. Примените -indvars, -licm, -loop-unroll.
3. Постройте CFG.
4. Укажите в чем отличие while от do-while в IR (задание 2.15.1).

2.17 Цикл for

Авдюков Богдан Алексеевич, АП-327

Гарес Дмитрий Александрович, АВТ-314

Княгинина Элеонора Александровна, АВТ-314

Зенцов Вадим Александрович, АВТ-313

Пример кода:

```
int main() {  
    int sum = 0;  
    for (int i = 0; i < 10; i++) {  
        sum += i;  
    }  
    return sum;  
}
```

Задания:

1. Получите IR для -O0.
2. Получите IR для -O2. Раскрыт ли цикл?
3. Примените -unroll-count=4 и сравните.
4. Постройте CFG.
5. Сделайте вывод об оптимизации циклов в LLVM.

Дополнительное задание. Локальные оптимизации синтаксических конструкций на промежуточном представлении

Для выбранной синтаксической конструкции (в соответствии с вашим вариантом КР / РГР):

1. Построить абстрактное синтаксическое дерево (AST) конструкции (из лабораторной работы 5).
2. Сгенерировать промежуточное представление (IR) для данной конструкции в виде трехадресного кода (ТАС) или списка виртуальных инструкций или канонической строковой формы (упрощённый вариант для одной строки).
3. Реализовать **две** локальные оптимизации. Каждая оптимизация должна преобразовывать IR к эквивалентному, но более простому или каноническому виду.
4. Продемонстрировать работу оптимизаций на конкретных примерах: показать «входной IR» и «выходной IR» после применения каждой оптимизации.

Требования к реализации:

- Оптимизации должны быть локальными, то есть работать внутри одной конструкции, без анализа других частей программы.
- Преобразование должно сохранять семантику исходной конструкции.
- Допускается как упрощение кода (удаление лишних скобок, необязательных атрибутов), так и канонизация (приведение к стандартному виду, упорядочивание элементов).
- Для константных выражений (если они есть в конструкции) должна выполняться свертка (вычисление на этапе компиляции).
- Для каждого типа оптимизации привести описание, тестовые примеры (скриншоты), а также описание реализации оптимизации в виде обобщенных блок-схем.

Примеры локальных оптимизаций (можно выбрать не из списка):

Оптимизация	Исходный IR	Преобразованный IR	Пояснение
Удаление лишних скобок	<code>return x + (y * z);</code>	<code>return x + y * z;</code>	Скобки не влияют на порядок операций
Свертка констант	<code>return 2 + (3 * 4);</code>	<code>return 14;</code>	Вычислено на этапе компиляции
Упрощение типов	<code>int sum(int x, int y)</code>	<code>auto sum(int x, int y)</code>	Тип возврата выводится автоматически
Упорядочивание полей структуры	<code>struct exam { char c; int i; short s; };</code>	<code>struct exam { int i; short s; char c; };</code>	Канонический порядок полей (по размеру)
Упрощение комплексного числа	<code>std::complex<double> z(5.0, 0.0);</code>	<code>double z = 5.0;</code>	Мнимая часть равна нулю, то есть можно заменить на вещественное
Удаление дубликатов в списке	<code>let arr = [1, 2, 2, 3, 3, 3];</code>	<code>let arr = [1, 2, 3];</code>	Для множества дубликаты не имеют смысла
Удаление неиспользуемого параметра	<code>func f(a, b, c) { return a + b; }</code>	<code>func f(a, b) { return a + b; }</code>	Параметр c не используется в теле
Преобразование while в for	<code>int i = 0; while (i < 10) { i++; }</code>	<code>for (int i = 0; i < 10; i++) { }</code>	Каноническая форма цикла со счётчиком

Замена if-else на тернарный	<pre>if (a > b) { max = a; } else { max = b; }</pre>	<pre>max = (a > b) ? a : b;</pre>	Более краткая форма
Упрощение лямбды без параметров	<pre>f = lambda: 40 + 2</pre>	<pre>f = lambda: 42</pre>	Упрощённая форма для лямбды-константы
Упорядочивание ключей словаря	<pre>let dict = {"b": 2, "a": 1, "c": 3};</pre>	<pre>let dict = {"a": 1, "b": 2, "c": 3};</pre>	Канонический порядок (по ключу)
Свертка арифметики с нулём	<pre>int x = a * 0;</pre>	<pre>int x = 0;</pre>	Результат всегда ноль
Нормализация научной нотации	<pre>double d = 0.000123e4;</pre>	<pre>double d = 1.23;</pre>	Упрощение числа